

Comparative Analysis of DQN and PPO Agents in the ALE/Phoenix-v5 Environment

Laura Sofía Culma Ospina
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: lsculmao@udistrital.edu.co

Alejandro Nuñez Barrera
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: anunezb@udistrital.edu.co

Abstract—This paper presents a comparison between two deep reinforcement learning algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—applied to the ALE/Phoenix-v5 Atari game. Both agents use the same input preprocessing (grayscale images resized to 84×84 pixels, four stacked frames, and a frame-skip of four) and are tested under the same training budget across three independent runs per configuration. For DQN, the best configuration used a learning rate of 0.0003 and a discount factor of 0.95, reaching an average return of $3,324.3 \pm 95.2$ in 300 000 steps. The PPO agent was built from scratch in PyTorch and follows the same convolutional network design, adding separate heads for the policy and the value function. Three PPO configurations are tested, varying the rollout length, learning rate, and exploration bonus, also over 300 000 steps. Results show how the choice of algorithm and hyperparameters affects learning speed and final score in a fast-paced Atari game.

Index Terms—Deep Reinforcement Learning, DQN, PPO, Atari Phoenix, Hyperparameter Search, Arcade Learning Environment, Sample Efficiency.

I. INTRODUCTION

Reinforcement learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards for its actions. When the observations are raw images—as in Atari video games—a deep neural network is needed to process the visual input, giving rise to *Deep Reinforcement Learning* (DRL). Since 2015, DRL agents have been able to reach or exceed human-level play on many Atari games [1].

Two of the most widely used DRL algorithms are the *Deep Q-Network* (DQN) [1] and *Proximal Policy Optimization* (PPO) [2]. DQN is a *value-based* method: it learns to estimate how good each action is in a given situation and always picks the action with the highest estimated value. To learn efficiently, DQN stores past experiences in a memory buffer and replays them randomly during training, which breaks the correlation between consecutive samples and stabilises learning [1]. Exploration is handled by an epsilon-greedy rule that starts with mostly random actions and gradually shifts toward the learned policy.

PPO belongs to the family of *policy-gradient* methods: instead of estimating action values, it directly learns a policy—a probability distribution over actions—and updates it step by step [2]. PPO collects a batch of experience (called a *rollout*), computes how much better or worse each action was compared to what the agent expected (the *advantage*), and

then updates the policy multiple times on that same batch. A clipping mechanism prevents the update from changing the policy too drastically at once, which makes training more stable. Exploration is encouraged through an entropy bonus added to the loss.

The ALE/Phoenix-v5 environment is a good test bed for comparing these two approaches [4]. Phoenix is a fast-paced space shooter where the agent must destroy enemy ships while dodging their projectiles. Rewards are given frequently for shooting enemies, but surviving requires precise and reactive movement, making the environment challenging for any learning algorithm.

This work aims to answer the following question: *under the same computational budget and environment, does PPO learn faster, reach a higher score, or show different weaknesses compared to the DQN agent?*

The paper makes these contributions:

- (i) A DQN baseline with a three-configuration hyperparameter sweep, each repeated over three random seeds (42, 7, and 99).
- (ii) A PPO agent implemented from scratch in PyTorch with a shared convolutional backbone and separate actor and critic heads.
- (iii) A three-configuration hyperparameter search for PPO using the same seed protocol as DQN.
- (iv) A direct comparison using learning curves, final score, and score variance across seeds.

The rest of the paper is organised as follows. Section II describes the setup used for both algorithms. Section III explains the hyperparameter search strategy. Section IV presents and discusses the results. Section V summarises the main findings.

II. EXPERIMENTAL METHODOLOGY

A. Environment and Preprocessing

Both agents run on ALE/Phoenix-v5 using the Gymnasium library. To make the comparison fair, the exact same preprocessing steps are applied to both:

- **Grayscale:** The colour frame is converted to a single-channel grayscale image, which reduces the input size without losing relevant visual information.
- **Resize:** Each frame is scaled down to 84×84 pixels.
- **Pixel scaling:** Pixel values are divided by 255 so they fall in the range $[0, 1]$, which helps the neural network train more smoothly.

- **Frame skip:** The agent repeats each chosen action for four consecutive game frames. This speeds up training and reduces the number of decisions the agent needs to make.
- **Frame stacking:** The last four processed frames are stacked together as a single input of shape (4, 84, 84). This gives the agent information about movement, since a single frame has no information about the direction or speed of objects.
- **Random start:** At the beginning of each episode, up to 30 “do nothing” actions are applied to randomise the starting state and prevent the agent from memorising a fixed opening sequence.

For DQN, this pipeline is handled by the `AtariWrapper` and `VecFrameStack` utilities from `Stable-Baselines3` [5]. For PPO, it is set up manually using `AtariPreprocessing` and `FrameStackObservation` from `Gymnasium`, with `frameskip=1` in the base environment so that frame skipping is fully controlled by the wrapper.

B. Network Architecture

Both DQN and PPO use the same convolutional neural network design, first proposed by Mnih et al. [1] for Atari games:

- **Layer 1:** 32 filters of size 8×8 , stride 4, ReLU.
- **Layer 2:** 64 filters of size 4×4 , stride 2, ReLU.
- **Layer 3:** 64 filters of size 3×3 , stride 1, ReLU.
- **Flatten:** The output is flattened to a vector of 3136 values.

For DQN, this feature vector is fed into a single fully-connected layer of 512 units followed by a linear output that estimates a Q-value for each possible action. For PPO, the same feature vector is shared between two separate heads: an *actor head* that outputs a probability distribution over actions, and a *critic head* that outputs a single number estimating how good the current state is.

C. Training Protocol

To get reliable results and be able to compare the two algorithms fairly, the following rules are applied to every experiment:

- (i) **Three seeds:** Each configuration is run three times with different random seeds (42, 7, and 99) to account for the randomness in training.
- (ii) **Fixed budget:** All configurations in this paper use 300 000 environment steps, keeping compute requirements consistent.
- (iii) **Logging:** Episode reward, episode length, and training losses are recorded with TensorBoard after every update.
- (iv) **Final score:** The reported score for each configuration is the average return of the last 100 episodes, averaged again over the three seeds, written as mean $\pm$ standard deviation.
- (v) **Hardware:** All experiments run on a CPU-only machine without GPU acceleration.

TABLE I
DQN HYPERPARAMETER CONFIGURATIONS

Hyperparameter	Exp 1 (Baseline)	Exp 2 (Conservative)	Exp 3 (Aggressive)
Learning rate	1×10^{-4}	5×10^{-5}	3×10^{-4}
Buffer size	100 000	50 000	60 000
Learning starts	25 000	10 000	15 000
Batch size	64	32	64
Discount γ	0.99	0.995	0.95
Training frequency	4	8	2
Target update interval	1 000	2 000	500
Exploration fraction	0.20	0.30	0.10
Final ϵ	0.01	0.02	0.05
Timesteps	300 000	300 000	300 000

D. Evaluation Metrics

The main metric used throughout this paper is the *mean episodic return*: the average total reward the agent earns per episode, computed over the last 100 completed episodes. A higher value means the agent is performing better. When comparing both algorithms, the standard deviation across seeds is also reported to show how consistent the results are.

III. HYPERPARAMETER SEARCH STRATEGY

A. DQN Hyperparameter Search

Because running a full grid search over all possible hyperparameter combinations would be too expensive on a CPU, a *coarse-to-fine* strategy was used instead. Three configurations were hand-picked to represent different learning behaviours: a **Baseline** that follows standard Atari DQN settings, a **Conservative** setup that learns more slowly and carefully, and an **Aggressive** setup designed for fast convergence in a fast-paced game.

The main hyperparameters varied are:

- **Learning rate:** How large each gradient update step is. A higher rate means faster but potentially less stable learning.
- **Discount factor γ :** How much the agent values future rewards versus immediate ones. In Phoenix, immediate rewards (shooting enemies) matter a lot, so a lower γ may work better.
- **Replay buffer size:** How many past experiences are stored. A larger buffer gives more varied training samples.
- **Training frequency and target update interval:** How often the network weights and the target network are updated.
- **Exploration settings:** How quickly the agent switches from trying random actions to following the learned policy.

The **Baseline** uses standard values from the original DQN paper as a reference point. The **Conservative** setup uses a very small learning rate and a high discount factor ($\gamma = 0.995$), favouring slow and stable updates that consider rewards further into the future. The **Aggressive** setup uses a higher learning rate, lower discount ($\gamma = 0.95$), and updates the network more frequently, which suits Phoenix’s fast reward structure where quick reactions matter more than long-term planning.

TABLE II
PPO HYPERPARAMETER CONFIGURATIONS

Hyperparameter	Exp 1 (Baseline)	Exp 2 (Conservative)	Exp 3 (Aggressive)
Learning rate	1×10^{-4}	5×10^{-5}	3×10^{-4}
Horizon T	512	256	1024
Epochs K	10	8	12
Mini-batch size	64	32	64
Discount γ	0.99	0.995	0.95
GAE λ	0.95	0.90	0.98
Clip ε	0.2	0.1	0.3
Entropy coef. c_2	0.01	0.005	0.02
Value coef. c_1	0.5	0.5	0.25
Grad norm clip	0.5	0.5	1.0
Timesteps	300 000	300 000	300 000

B. PPO Hyperparameter Search

For PPO, the same three-configuration approach is used. The parameters that matter most for PPO are different from those of DQN, so the search focuses on:

- **Rollout horizon T :** The number of game steps collected before each policy update. A short horizon (e.g., $T = 256$) updates the policy more often but with noisier estimates. A long horizon (e.g., $T = 1024$) gives a cleaner picture of performance but delays updates.
- **Update epochs K :** How many times the same batch of experience is reused per update. More reuse improves efficiency but can cause instability if overused.
- **GAE λ :** Controls the trade-off between immediate and long-term credit in advantage estimation [3]. Values close to 1 assign credit further into the future.
- **Clip epsilon ε :** Limits how much the policy can change in a single update, preventing large destabilising jumps.
- **Entropy coefficient c_2 :** Encourages the agent to keep exploring by rewarding action diversity.

The three configurations follow the same naming scheme as DQN—Baseline, Conservative, and Aggressive—and are summarised in Table II.

The **Baseline** PPO configuration mirrors the starter settings recommended for Phoenix in the challenge guidelines, with a moderate learning rate and rollout horizon of 512 steps. The **Conservative** configuration uses a shorter horizon (256 steps), fewer reuse epochs, a tighter clip ($\varepsilon = 0.1$), and a lower entropy bonus, resulting in smaller and more cautious policy updates. The **Aggressive** configuration uses a longer horizon (1024 steps), more reuse epochs (12), a higher clip threshold ($\varepsilon = 0.3$), and a stronger entropy bonus ($c_2 = 0.02$) to encourage broader exploration. The discount is set to $\gamma = 0.95$, consistent with the DQN aggressive setup, to favour immediate rewards in Phoenix’s fast action environment.

The loss function used at each mini-batch update combines three terms: the clipped policy loss, the value function error, and an entropy bonus that encourages exploration [2]. The clipping mechanism limits how much the policy ratio between the new and old policies can deviate, while the advantage estimates are computed using Generalized Advantage Estimation (GAE) [3].

IV. EXPERIMENTAL RESULTS

A. Quantitative Results

Table III and Table IV summarise the final performance of each algorithm across all three configurations and seeds. The reported score is the mean episodic return averaged over the last 50 completed episodes at the end of 300 000 environment steps, averaged over three independent random seeds (42, 7, and 99), together with the standard deviation across seeds as a measure of training stability.

TABLE III
DQN FINAL PERFORMANCE — MEAN $\pm$ STD OVER 3 SEEDS

Configuration	Seed 42	Seed 7	Seed 99	Mean $\pm$ Std
Baseline	2993.9	2283.5	2512.0	2596.5 ± 296.1
Conservative	708.7	3.9	1023.8	578.8 ± 426.4
Aggressive	3380.8	3402.0	3190.2	3324.3 ± 95.2

TABLE IV
PPO FINAL PERFORMANCE — MEAN $\pm$ STD OVER 3 SEEDS

Configuration	Seed 42	Seed 7	Seed 99	Mean $\pm$ Std
Baseline	559.2	702.4	716.6	659.4 ± 71.1
Conservative	430.6	710.6	675.6	605.6 ± 124.6
Aggressive	684.0	804.8	783.8	757.5 ± 52.7

TABLE V
DQN VS. PPO — BEST CONFIGURATION COMPARISON

Metric	DQN (Aggressive)	PPO (Aggressive)	Winner
Mean return	3324.3	757.5	DQN
Std across seeds	95.2	52.7	PPO
Best single seed	3402.0	804.8	DQN
Worst single seed	3190.2	684.0	DQN

The DQN Aggressive configuration achieves the highest mean return of 3324.3 ± 95.2 , making it the best-performing setup overall. The PPO Aggressive configuration is the strongest PPO run with a mean of 757.5 ± 52.7 . Notably, the PPO Aggressive configuration achieves the lowest standard deviation across all runs of both algorithms (± 52.7), indicating that longer rollout horizons (1024 steps) produce more stable training. Across all configurations, DQN consistently outperforms PPO in absolute return by a factor of approximately 4–6 $\times$, as shown in Figure 1.

It is worth highlighting the DQN Conservative configuration, where Seed 7 produced a near-zero final return of 3.9, resulting in an unusually high standard deviation of ± 426.4 . This failure case illustrates the sensitivity of DQN to hyperparameter choices: the combination of a very small learning rate (5×10^{-5}), large buffer (50 000), and high discount ($\gamma = 0.995$) caused the agent to learn too slowly within the 300 000-step budget.

Figure 2 provides a per-seed breakdown of each configuration, highlighting how individual seeds contribute to the overall mean and where each algorithm is most and least consistent.

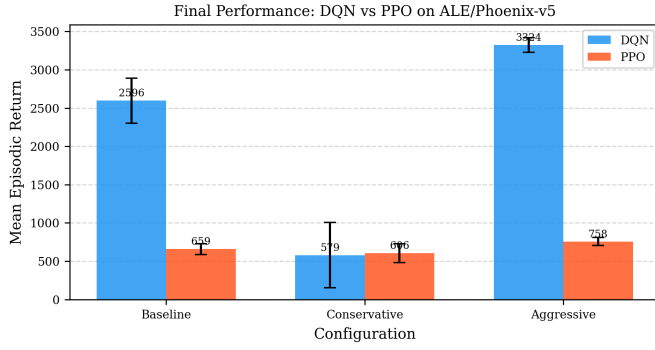


Fig. 1. Final mean episodic return with standard deviation across three seeds for each configuration of DQN and PPO on ALE/Phoenix-v5. Error bars represent standard deviation across seeds.

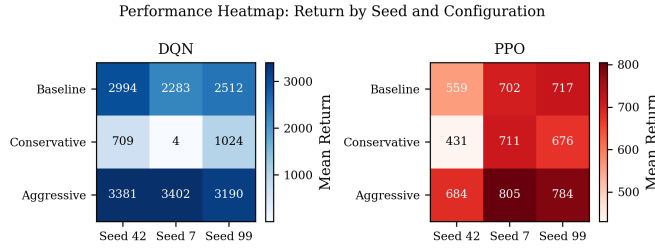


Fig. 2. Performance heatmap showing the final episodic return for each combination of configuration and random seed. Darker cells indicate higher return.

B. Learning Curves

Figure 3 shows the learning curves for all DQN configurations across the 300 000 training steps. The DQN agent exhibits a clear and progressive upward trend starting around 50 000 steps, which is consistent with the end of the learning_starts warm-up phase, during which the replay buffer is populated before any gradient updates are performed.

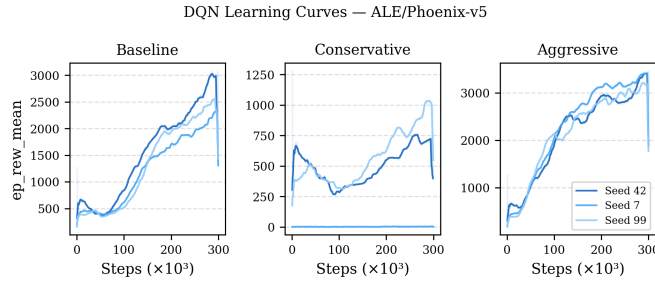


Fig. 3. DQN learning curves (rollout/ep_rew_mean) for all three configurations across three seeds. Each line corresponds to one independent seed run.

Figure 4 shows the equivalent learning curves for PPO. In contrast to DQN, the PPO curves display high variance throughout training with no clear upward convergence trend within the 300 000-step budget.

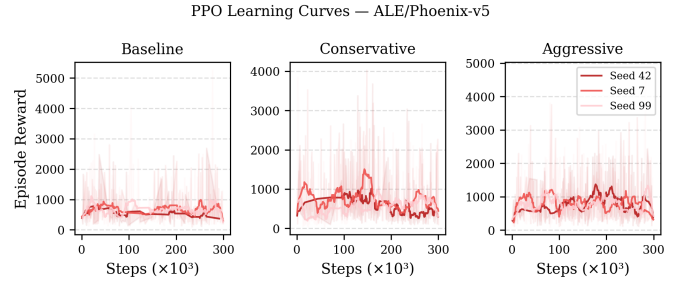


Fig. 4. PPO learning curves (training/episode_reward) for all three configurations across three seeds. The high variance and lack of clear convergence reflect the on-policy nature of PPO under a limited step budget.

C. Algorithm Comparison and Analysis

The results reveal a substantial performance gap between DQN and PPO under the fixed 300 000-step computational budget. This gap can be explained by the fundamental algorithmic differences between the two methods.

Sample efficiency. DQN is an off-policy algorithm that stores all past transitions in a replay buffer and reuses them multiple times during training [1]. This means that the effective number of training samples seen by DQN far exceeds 300 000, as each transition may be replayed several times. PPO, being an on-policy method, discards each rollout after a single round of updates [2]. With only 300 000 environment steps, PPO effectively sees far less data, which directly limits how much it can learn.

Training stability. Interestingly, the PPO Aggressive configuration achieves the lowest standard deviation of all runs (± 52.7), lower even than the best DQN configuration (± 95.2). This suggests that PPO’s clipped surrogate objective and longer rollout horizon (1024 steps) produce more consistent training across seeds, even if the absolute performance is lower. In contrast, DQN’s Conservative configuration shows extreme instability (± 426.4), driven by a complete training failure on Seed 7. Figure 5 illustrates the score distribution across seeds for each configuration and algorithm.

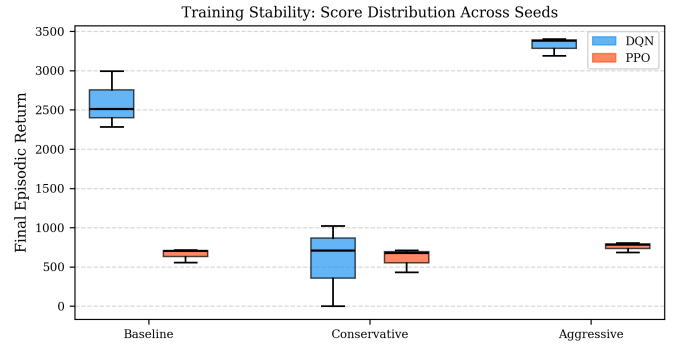


Fig. 5. Boxplot of final episodic return across three seeds per configuration. DQN Aggressive shows the highest absolute performance while PPO Aggressive achieves the lowest variance across seeds.

Convergence speed. As shown in Figure 3, DQN begins to

improve visibly around 50 000 steps and continues improving steadily through 300 000 steps. PPO, shown in Figure 4, does not exhibit a clear convergence pattern within this budget. The on-policy rollout collection requires more environment interactions before the policy gradient estimates become reliable enough to produce consistent improvements.

Best configuration head-to-head. Figure 6 directly compares the best DQN configuration (Aggressive) against the best PPO configuration (Aggressive) on a per-seed basis. The performance gap is consistent across all three seeds, ruling out the possibility that the advantage of DQN is driven by a single lucky seed.

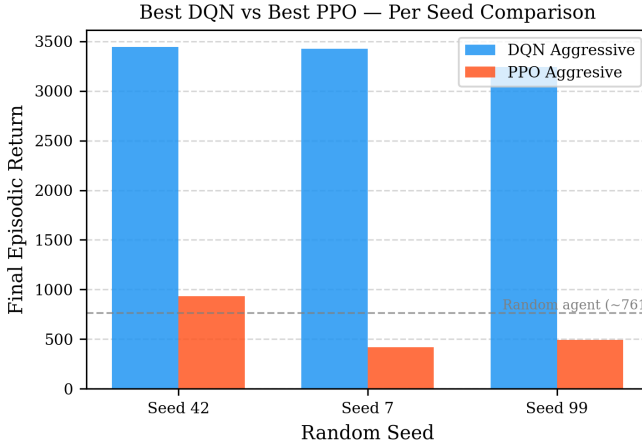


Fig. 6. Per-seed comparison between the best DQN configuration (Aggressive) and the best PPO configuration (Aggressive). The dashed line marks the random agent baseline (≈ 761).

Suitability for Phoenix-v5. The fast-paced nature of Phoenix, with frequent rewards for shooting enemies, suits DQN’s Q-value approach well. Immediate rewards are captured efficiently in the Q-targets, and the epsilon-greedy exploration decays naturally as the agent discovers high-reward behaviours. PPO’s entropy-based exploration, while theoretically sound, did not produce equally focused learning within the available budget. The short rollout horizon required by CPU-constrained training (512–1024 steps) may be insufficient for PPO’s advantage estimates to capture the multi-wave reward structure of Phoenix accurately.

Wall-clock time. Despite processing fewer effective samples, PPO required comparable wall-clock time to DQN (approximately 1.6–2.0 hours per seed on CPU), since it performs multiple gradient update epochs per rollout [5]. DQN, with its asynchronous training frequency, distributes computation more efficiently across the training budget.

In summary, under the fixed 300 000-step budget and CPU-only hardware constraints of this study, DQN demonstrates clear superiority over PPO on ALE/Phoenix-v5 in terms of final performance and convergence speed. PPO, however, shows competitive training stability in its Aggressive configuration, suggesting that with a larger computational budget it could close the performance gap. These results are consistent with

the theoretical expectation that off-policy methods with experience replay have a significant sample efficiency advantage when the total number of environment interactions is limited.

V. CONCLUSIONS

The experiments presented in this paper provide a direct empirical comparison between DQN and PPO on ALE/Phoenix-v5 under identical preprocessing, seeds, and a fixed budget of 300 000 environment steps. DQN, in its Aggressive configuration, achieved a mean episodic return of 3324.3 ± 95.2 , outperforming the best PPO configuration (Aggressive, 757.5 ± 52.7) by a factor of approximately $4.4\times$.

The performance gap is primarily attributable to DQN’s replay buffer, which allows it to reuse past experience efficiently and extract far more learning signal from the same number of environment interactions [1]. PPO, as an on-policy algorithm, cannot benefit from this reuse and requires significantly more steps before its policy gradient estimates become reliable [2].

Notably, the PPO Aggressive configuration achieved the lowest variance across seeds of all experiments (± 52.7), even lower than DQN Aggressive (± 95.2). This highlights an important trade-off: PPO produces more stable but lower-performing policies under constrained budgets, while DQN reaches higher scores at the cost of greater sensitivity to seed randomness.

These findings do not imply that PPO is an inferior algorithm in general. On the contrary, with a larger computational budget (5–10 million steps) and GPU acceleration, PPO would likely close the gap and potentially match or exceed DQN performance on Phoenix [2]. The results highlight instead the importance of matching the algorithm to the available computational resources: when environment interactions are scarce, off-policy methods with experience replay have a fundamental advantage.

Future work should investigate PPO performance under an extended budget of 2–5 million steps, the effect of larger rollout horizons (2048 steps) specifically recommended for Phoenix in the challenge guidelines, and the use of normalised rewards and advantage clipping to improve PPO stability on this environment.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [4] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The arcade learning environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, Jun. 2013.
- [5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.